Privilege OpenAPI MCP on Kubernetes

Working Pattern and Remaining Gap

DATE
2026-10-01

PURPOSE
Explain the OpenAPI-to-MCP path in this repository and compare it with the supplied Kubernetes deployment ask.

Summary

WORKING PATTERN, NOT DYNAMIC AUTO-DEPLOYMENT

The repository has a Kubernetes-native way to run Ping's OpenAPI MCP adapter: declare it as a static sidecar in the Privilege gateway Pod and register that sidecar as an MCP Server application. This lets the gateway reach the adapter over the Pod's shared localhost network. The gateway process does not need a Docker or containerd socket, nor permission to create Kubernetes resources.

This is a working deployment pattern for a configured adapter. It is not a general controller that watches each generated app descriptor and creates a Deployment and Service for it. The supplied PDF's Petstore case therefore remains a separate, broader automation gap. The evidence in this repository is from the ai-demo-cmuir / ping-devops-curtismuir setup; the PDF describes ping-devops-tarunmadiraju, so the two environments should not be treated as the same live deployment.

What was failing

The original openapi2 Agentic App used the Privilege-hosted mcp/openapi image. Investigation recorded that this image runs /openapimcp without the mcp-shim present in working catalog images. The adapter itself accepted MCP initialize and generated tools, but its GET /mcp response was 405. The Privilege gateway's normal MCP Server discovery path uses a POST initialize; the OpenAPI app type followed a different, Privilege-managed container path and did not discover tools in this setup. See the investigation in CURRENT-CONFIGURATION.md, lines 223–275.

The PDF reports a related but distinct failure: Privilege wrote a generated runtime descriptor for Petstore, but no Kubernetes workload appeared. It also records that the gateway's service account cannot create Pods or Deployments. The repo's workaround avoids asking the gateway process to launch containers at runtime.

How the repository path works

1

Keep the OpenAPI-described API reachable in the gateway Pod. The demo_mcp_banking_rest service runs as an extraContainers sidecar on port 8082. It exposes a small self-contained /users API and /openapi/simple-users.json for a focused discovery probe, as well as proxy routes for the banking REST API. The banking proxy forwards the API key to the resource server; the OpenAPI document is fetched without that key. See server.js and its route tests.

2

Run the OpenAPI MCP adapter as another static sidecar. The checked-in Helm values define mcp-openapi-banking with image public.ecr.aws/n2z2g8w6/mcp/openapi:latest, listener port 8080, and these settings:

```
OPENAPIMCP_ENDPOINT=http://localhost:8082
OPENAPIMCP_SPEC_URL=http://localhost:8082/openapi/simple-users.json
OPENAPIMCP_TOOL_MODE=both
```

OPENAPIMCP_ENDPOINT is the REST API base URL, so it has no /mcp suffix. The endpoint and spec are on the demo_mcp_banking_rest sidecar in the same Pod. The values are in sidecars.values.yaml.

3

Register the adapter through the gateway's supported MCP Server path. The recorded console configuration uses application type MCP Server, name banking-openapi, and backend http://localhost:8080/mcp. The gateway can then POST initialize to the adapter and discover tools through its standard MCP Server flow. The specific app registration is manual configuration; it is not generated from each OpenAPI descriptor. See CURRENT-CONFIGURATION.md, lines 223–275.

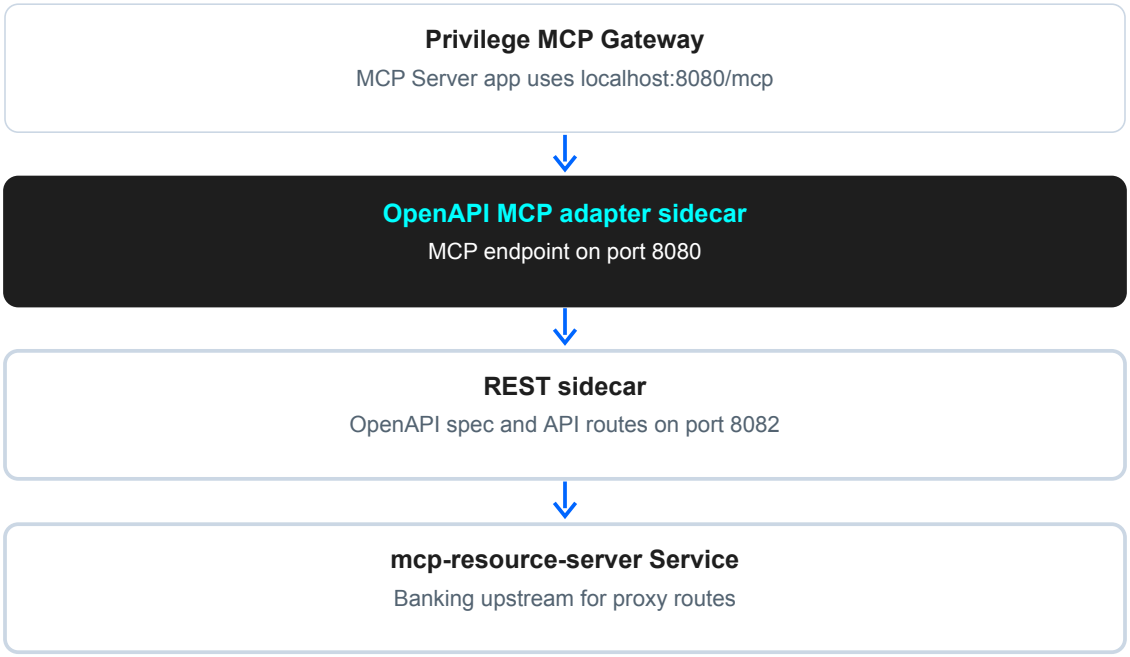
4

Use a TCP readiness probe. The adapter's GET /mcp returns 405, while MCP initialization is a POST. An HTTP GET probe would report a healthy, running adapter as unready, so the sidecar uses tcpSocket on port 8080 instead.

5

Deploy through Helm, not from inside the gateway. The vendored chart adds configured extraContainers to the gateway Pod template. Kubernetes schedules them as part of the Pod; containers share its network namespace, which makes the configured localhost URLs work. On Helm upgrades, extraContainers is a list and must be supplied in full so existing sidecars are not removed. The chart hook is a local change and is lost if the chart is re-vendored without carrying it forward. See deployment.yaml and the upgrade guidance at the top of sidecars.values.yaml.

REQUEST FLOW



What is verified in the repository

- The REST sidecar serves the local listUsers example and OpenAPI 3.0.3 spec. Its unit tests cover those endpoints and the API-key-protected banking proxy. These tests do not run the OpenAPI MCP image or a Kubernetes deployment.
- The operational notes record a live end-to-end check of the banking REST sidecar reaching seeded accounts through the resource-server Service and API-key authentication (AGENTLESS-CONFIGURATION.md, lines 77–114).
- The OpenAPI adapter investigation records successful POST /mcp initialize and tool generation when the image was run with a valid OpenAPI configuration; it also records the 405 GET behavior that motivates the TCP probe.
- The checked-in OpenAPI sidecar configuration currently points to the self-contained Simple User API at /openapi/simple-users.json. It is a controlled adapter/discovery probe, not the Petstore URL from the PDF. The current file's BANKING_UPSTREAM_URL also differs from the cross-namespace target recorded in the operational notes. The local /users probe does not depend on that upstream, so the committed values alone do not establish that the present banking proxy target is correct.

The later banking-mcp entry in CURRENT-CONFIGURATION.md, lines 277–310 is a direct MCP Server integration. It may serve the banking demo, but it does not exercise OpenAPI conversion or solve generated-app deployment.

Answer to the engineering ask

The repository demonstrates that the OpenAPI MCP runtime can run on Kubernetes without granting the Privilege gateway Pod Kubernetes create permissions: deploy a preconfigured runtime sidecar with Helm, give it the spec and REST endpoint, and register its MCP endpoint with the gateway.

It does not yet provide automatic Kubernetes deployment for arbitrary generated OpenAPI apps. In particular, the code does not watch the generated descriptor, create a per-app Deployment and Service, or update the app's backend address to the resulting Kubernetes Service URL. Those capabilities are required to close the PDF's Petstore workflow. The static sidecar pattern is useful where the OpenAPI app is known at deploy time; it does not scale to dynamic app creation by itself.

Source files

- privilege/CURRENT-CONFIGURATION.md — observed failure, adapter behavior, MCP Server registration, and later banking integration status.
- privilege/AGENTLESS-CONFIGURATION.md — original OpenAPI REST sidecar configuration and live REST-path verification.
- pingone-privgateway-helm-main/agentless/sidecars.values.yaml — checked-in sidecar image, environment, port, and readiness settings.
- pingone-privgateway-helm-main/agentless/agentless-mcpgw/templates/deployment.yaml — Helm template that injects extraContainers into the gateway Pod.
- demo_mcp_banking_rest/server.js and demo_mcp_banking_rest/test/server.test.js — local OpenAPI probe and REST proxy implementation.
- demo_mcp_resource_server/openapi/banking-rest.openapi.json and demo_mcp_resource_server/src/index.ts — banking REST OpenAPI document and server route.